The Journal of Open Source Software

py_neuromodulation: Signal processing and decoding for neural electrophysiological recordings

Timon Merk [1¶], **Antonio Brotons** [1], **Samed R. Vossberg** [1], **Richard M. Köhler** [1], **Thomas S. Binns** [1,2,3], **Alessia Cavallo** [1,2], **Elisa Garulli** [4], **Ashley Walton** [5], **Jojo Vanhoecke** [1,2], **R. Mark Richardson** [5], and **Wolf-Julian Neumann** [1,2,3]

1 Movement Disorders Unit, Charité - Universitätsmedizin Berlin, Berlin, Germany **2** Bernstein Center for Computational Neuroscience Berlin, Berlin, Germany **3** Einstein Center for Neurosciences Berlin, Berlin, Germany **4** Department of Neurology with Experimental Neurology, Charité – Universitätsmedizin Berlin, Berlin, Germany **5** Department of Neurosurgery, Massachusetts General Hospital, Harvard Medical School, Boston, Massachusetts, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 31 March 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Invasive brain signal decoding can revolutionize the clinical utility of neurotechnological therapies. Potential signal sources stem from electroencephalography (EEG), electrocorticography (ECoG) and local field potentials (LFP) recorded from deep brain stimulation (DBS) electrodes. The application of machine learning methods to these signals requires pre-processing and feature extraction – complex analyses, often lacking standardization and clear documentation. Here, we introduce py_neuromodulation, a toolbox designed for standardized signal processing, feature extraction, and decoding of electrophysiological data. All parameters are explicitly defined in dedicated settings and channel parameterization files. The framework processes both data streamed in real-time and from recordings on disk using the same pipeline. Additionally, a browser-based graphical user interface (GUI) enables intuitive usage and visualization of the processing pipeline without requiring any code modification.
By introducing py_neuromodulation, we aim to simplify and standardize the analysis of electrophysiological recordings, facilitating reproducibility and accessibility in the field of brain signal decoding. Our tool bridges the fields of neuroscience and neural engineering, providing machine learning and neurotechnology researchers with reproducible methods for the development of generalizable machine learning algorithms.

Introduction

Recently, deep learning foundation models have been presented that showed a performance leap on many clinically relevant downstream tasks in the field of brain signal decoding (Li et al., 2025; Yuan et al., 2024). Neural signal processing for machine learning and medical device engineering is a critical step for the advancement of the fields of neural decoding and brain computer interfaces. We therefore developed py_neuromodulation, a neural signal processing, feature calculation and decoding toolbox, that provides a standardized pipeline for both real-time and offline processing and decoding of neural recordings.
The analysis of electrophysiological recordings is commonly preceded by one or multiple pre-processing steps: resampling, notch-filtering, normalization, re-referencing, filtering (lowpass, highpass, bandpass) and artifact rejection (Cohen, 2014). Next, the features of interest are computed, often involving the transformation of neural signals into the spectral domain. Different spectral decomposition methods can be utilized, including fast Fourier transform (FFT), Welch's method, bandpass filtering and Hilbert transform, and others. Next to these

often used methods, more specific features such as waveform-shape (Cole & Voytek, 2017), separate periodic and aperiodic spectral components (Donoghue et al., 2020), burst features (Tinkhauser et al., 2018), and others can be computed.

There are a number of established toolboxes that enable the analysis of electrophysiological recordings. Popular toolboxes include Fieldtrip (Oostenveld et al., 2011) or Brainstorm (Tadel et al., 2011) in MATLAB and MNE-Python (Gramfort et al., 2013) in Python, among others. However, by introducing *py_neuromodulation*, we want to address specific challenges in the context of invasive *brain signal decoding*. For one, the variety of different processing parameters poses a challenge in selecting optimal methods for machine learning-based decoding, and second, hindering reproducibility since often the used parameters are not reported in scientific publications. To this end, we defined the parametrization files *settings.yaml* and *channels.tsv*, that include all required steps for reproducibility. Additionally, these allow for the construction of a unified pipeline of pre-processing, feature estimation and decoding. In this way, it is possible to assess the contribution of various pre-processing steps in combination with different feature modalities and machine learning model architectures to decoding performance.

Another pitfall stems from the fact that offline analysis of electrophysiology data often includes whole-recording normalization, non-causal filtering, or feature computation with test-set data leakage (Merk, Peterson, Köhler, et al., 2022), which limits the generalizability of findings to real-time applications. In py_neuromodulation, the data source is represented as a Python generator, and can be interchanged for online and offline processing, meaning that there is no difference in handling data streamed directly from a brain implant, or from a previously recorded dataset stored on disk. This enables offline training of generalizable models that are directly applicable to online applications.

Finally, electrophysiological research entails multiple technical and conceptual hurdles that pose substantial entry barriers for scientists without software development training. By providing an intuitive GUI for both parameter setting and real-time data and feature visualization, we aim to lower the domain knowledge required for entering this field.

Notably, py_neuromodulation has already been used in several scientific projects with various applications: ECoG and DBS-LFP based movement decoding (Cavallo et al., 2025; Merk, Peterson, Lipski, et al., 2022), movement intention classification (Köhler et al., 2024), seizure classification (Merk et al., 2023) and emotional valence decoding (Merk et al., 2023) in humans, as well as gait symptom estimation in rodents (ELGarulli, 2025).

Parametrization

A data source can be a two-dimensional array (channels x time), either passed directly as a NumPy array (Harris et al., 2020), as a Pandas DataFrame (team, 2023), or read from disk via mne.io.read_raw, or can be a LabStreamingLayer stream, which we connect to via mne-lsl (Figure 1). The parametrization **settings** and **channels** can be passed as optional parameters, or be omitted to fall back to reasonable default values.

The full parametrization options are described in the documentation in detail (Usage page). The main parameters are *sampling_rate_features_hz*, that defines in which interval features are computed, and the *segment_length_ms, that defines the signal duration used for computation of each feature. For pre-processing we provide several methods that can be reordered freely: notch_filtering, raw_resampling, re_referencing (common-average or individual bipolar re-referencing), raw_normalization and preprocessing_filter (lowpass, highpass, bandpass, bandstop). Additionally, a modifiable list of frequency ranges is passed, which is used for spectral feature computation. A list of currently implemented features is the following: `fft`, `stft`, `welch`, `bandpass_filter`, `raw_hjorth`, `return_raw`, `sharpwave_analysis`, `linelength`, `bursts`, bispectrum, fooof, coherence, nolds and mne_connectivity. The following post-processing methods can then be specified: feature_normalization and grid point interpolation (when coordinates and a cortical or subcortical grid are provided). For both raw data and feature normalization the following methods can be selected: `mean`, `median`, `zscore`, `zscore-median`, `quantile`, `power`, `robust` and `minmax`.

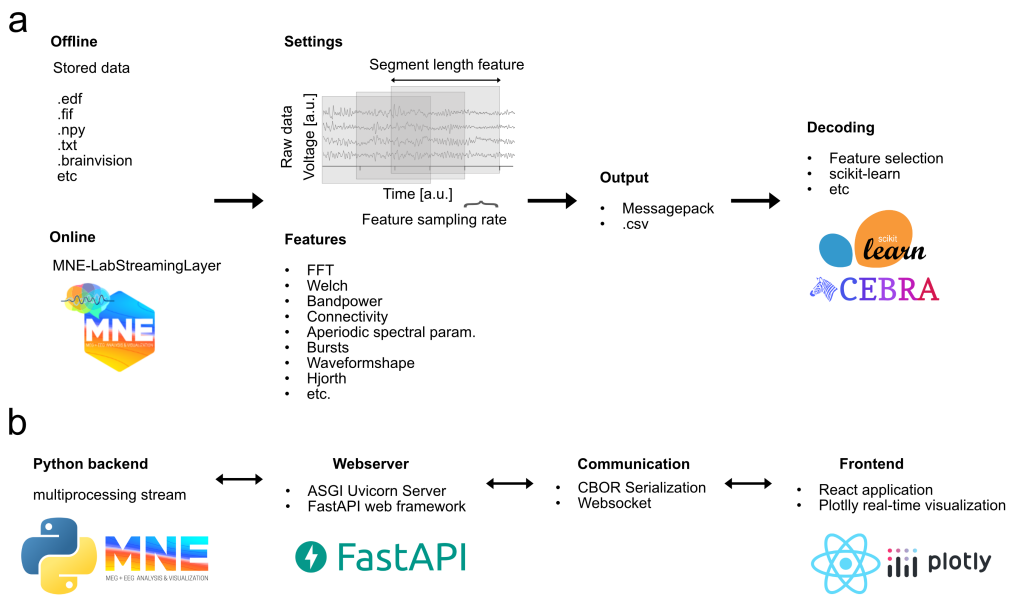

Figure 1: Schematic of py_neuromodulation .

Importantly, pre-processing and feature estimation can directly affect performances of decoding models. We added a **decode** module that can be parametrized with a scikit-learn compatible model, different cross-validation strategies, performance metrices, and dimensionality reduction methods, such that the computed features can be directly used to evaluate decoding performance within the same pipeline. A pre-trained model can also be passed to directly generate decoding outputs.

Graphical User Interface (GUI)

Finally, to simplify electrophysiological analysis and decoding, we added a React-based frontend application, which runs within an ASGI (Asynchronous Server Gateway Interface) uvicorn server and communicates through FastAPI. Through this frontend, the same settings as in the Python backend can be modified (Figure 2), a stream can then be selected from an offline file or via LabStreamingLayer, and raw data and features can be visualized in real time.

DRAFT

The Journal of Open Source Software

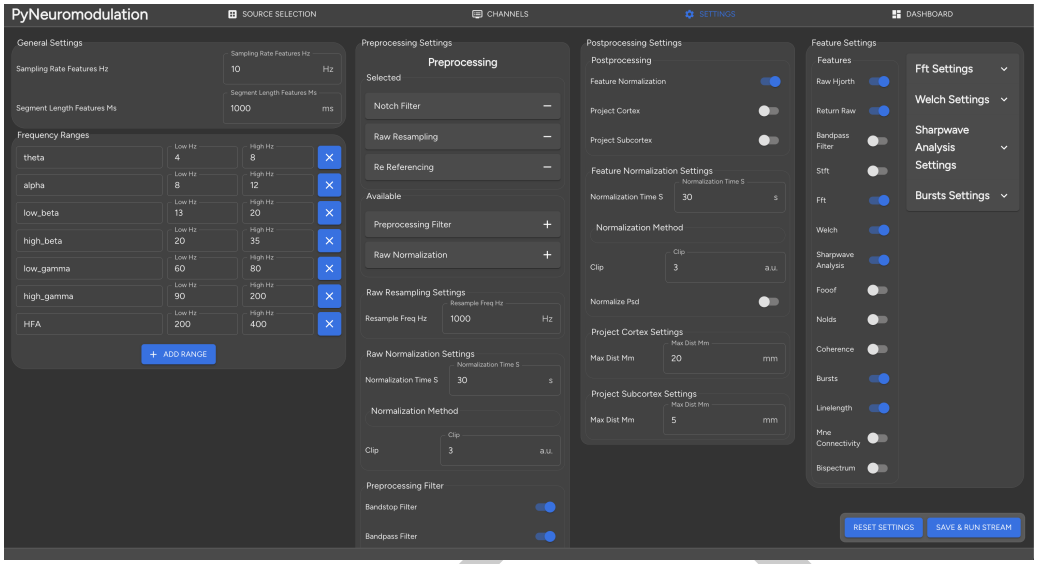

Figure 2: Frontend parametrization page representing `settings.yaml` configurations for pre-processing, feature estimation and post-processing.

Conclusion

In summary, py_neuromodulation provides a comprehensive, standardized framework for electrophysiological signal processing and neural decoding, addressing existing limitations in reproducibility and parameter documentation. Its unified pipeline supports both real-time and offline analyses, accompanied by an intuitive graphical interface to lower technical barriers in neural research. The successful use of py_neuromodulation across diverse applications highlights its potential as a broadly applicable tool for the analysis and machine-learning based decoding of electrophysiological data.

Acknowledgements

The study was funded by Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – Project-ID 424778381 and a US-German Collaborative Research in Computational Neuroscience (CRCNS) grant from the German Federal Ministry for Research and Education and NIH (R01NS110424). WJN received funding from the European Union (ERC, ReinforceBG, project 101077060).

References

Cavallo, A., Köhler, R. M., Busch, J. L., Habets, J. G. V., Merk, T., Zvarova, P., Vanhoecke, J., Marcelino, A. L. de A., Darcy, N., Herz, D. M., Schneider, G.-H., Faust, K., Yttri, E., Cagnan, H., Kühn, A. A., & Neumann, W.-J. (2025). Reinforcement of movement speed through speed-selective deep brain stimulation in Parkinson's disease. *Brain Stimulation: Basic, Translational, and Clinical Research in Neuromodulation*, *18*(1), 529. https://doi.org/10.1016/j.brs.2024.12.917

Cohen, M. X. (2014). *Analyzing Neural Time Series Data: Theory and Practice*. The MIT Press. https://doi.org/10.7551/mitpress/9609.001.0001

Cole, S. R., & Voytek, B. (2017). Brain Oscillations and the Importance of Waveform Shape. *Trends in Cognitive Sciences*, *21*(2), 137–149. https://doi.org/10.1016/j.tics.2016.12.008

DRAFT

Donoghue, T., Haller, M., Peterson, E. J., Varma, P., Sebastian, P., Gao, R., Noto, T., Lara, A. H., Wallis, J. D., Knight, R. T., Shestyuk, A., & Voytek, B. (2020). Parameterizing neural power spectra into periodic and aperiodic components. *Nature Neuroscience*, *23*(12), 1655–1665. https://doi.org/10.1038/s41593-020-00744-x

ELGarulli. (2025). *ELGarulli/neurokin*. https://github.com/ELGarulli/neurokin

Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, *7*. https://doi.org/10.3389/fnins.2013.00267

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Köhler, R. M., Binns, T. S., Merk, T., Zhu, G., Yin, Z., Zhao, B., Chikermane, M., Vanhoecke, J., Busch, J. L., Habets, J. G. V., Faust, K., Schneider, G.-H., Cavallo, A., Haufe, S., Zhang, J., Kühn, A. A., Haynes, J.-D., & Neumann, W.-J. (2024). Dopamine and deep brain stimulation accelerate the neural dynamics of volitional action in Parkinson's disease. *Brain*, *147*(10), 3358–3369. https://doi.org/10.1093/brain/awae219

Li, J., Chen, X., Shen, F., Chen, J., Liu, Y., Zhang, D., Yuan, Z., Zhao, F., Li, M., & Yang, Y. (2025). *Deep Learning-Powered Electrical Brain Signals Analysis: Advancing Neurological Diagnostics*. arXiv. https://doi.org/10.48550/arXiv.2502.17213

Merk, T., Köhler, R., Peterson, V., Lyra, L., Vanhoecke, J., Chikermane, M., Binns, T., Li, N., Walton, A., Bush, A., Sisterson, N., Busch, J., Lofredi, R., Habets, J., Huebl, J., Zhu, G., Yin, Z., Zhao, B., Merkl, A., … Neumann, W.-J. (2023). *Invasive neurophysiology and whole brain connectomics for neural decoding in patients with brain implants*. Research Square. https://doi.org/10.21203/rs.3.rs-3212709/v1

Merk, T., Peterson, V., Köhler, R., Haufe, S., Richardson, R. M., & Neumann, W.-J. (2022). Machine learning based brain signal decoding for intelligent adaptive deep brain stimulation. *Experimental Neurology*, *351*, 113993. https://doi.org/10.1016/j.expneurol.2022.113993

Merk, T., Peterson, V., Lipski, W. J., Blankertz, B., Turner, R. S., Li, N., Horn, A., Richardson, R. M., & Neumann, W.-J. (2022). Electrocorticography is superior to subthalamic local field potentials for movement decoding in Parkinson's disease. *eLife*, *11*, e75126. https://doi.org/10.7554/eLife.75126

Oostenveld, R., Fries, P., Maris, E., & Schoffelen, J.-M. (2011). FieldTrip: Open source software for advanced analysis of MEG, EEG, and invasive electrophysiological data. *Computational Intelligence and Neuroscience*, *2011*, 156869. https://doi.org/10.1155/2011/156869

Tadel, F., Baillet, S., Mosher, J. C., Pantazis, D., & Leahy, R. M. (2011). Brainstorm: A user-friendly application for MEG/EEG analysis. *Computational Intelligence and Neuroscience*, *2011*, 879716. https://doi.org/10.1155/2011/879716

team, T. pandas development. (2023). *Pandas-dev/pandas: Pandas*. Zenodo. https://doi.org/10.5281/zenodo.7979740

Tinkhauser, G., Torrecillos, F., Duclos, Y., Tan, H., Pogosyan, A., Fischer, P., Carron, R., Welter, M. L., Karachi, C., Vandenberghe, W., Nuttin, B., Witjas, T., Régis, J., Azulay, J. P., Eusebio, A., & Brown, P. (2018). Beta burst coupling across the motor circuit in Parkinson's disease. *Neurobiology of Disease*. https://doi.org/10.1016/j.nbd.2018.06.007

Yuan, Z., Shen, F., Li, M., Yu, Y., Tan, C., & Yang, Y. (2024). *BrainWave: A Brain Signal Foundation Model for Clinical Applications*. arXiv. https://doi.org/10.48550/arXiv.2402.10251

DRAFT